Retail Inventory Management Using Deep Reinforcement Learning

A reproducible comparison of DQN, stockout penalty tuning and an order up to heuristic

Final Project Report
Course Reinforcement Learning Theory
Project 5 Stochastic Retail Inventory Management

Submitted by
Virat Mankali
Roll number 23WU010227
B.Tech CSE (AI & ML)
Fourth Year • Panthers
Woxsen University

Report date 14 October 2026

Study outcome

The tuned (s, S) heuristic achieved the highest mean profit in both normal and stress testing. Increasing the DQN training stockout penalty tenfold reduced lost sales, but increased holding costs and reduced profit under the original economic objective.

This report documents the executed custom Gymnasium study. It explicitly identifies the deviation from the assignment's named or-gym environment and the pending recovery of standalone trained-model artifacts.

Abstract and report guide

This project investigates daily replenishment for a single retail product under stochastic demand and a two-day delivery lead time. A Deep Q-Network (DQN) learns discrete ordering actions from on-hand inventory, outstanding orders and time remaining. A second DQN is trained with a tenfold lost-sales penalty to test whether a more conservative objective improves service. Both are compared with a validation-selected (s, S) policy.

The experiment uses 30-day episodes, Poisson demand with a mean of 20 units per day, and 150,000 training steps per DQN run. Three seeds are used for each reward variant, giving six trained models. Every policy is evaluated on the same 100 independent normal-demand months and the same 100 independent stress months. Stress demand has a mean of 60 units on days 15–17. Selection uses a separate 50-month validation bank, and final test results are not used to tune policies.

Mean normal monthly profit is 22,615.980 for the heuristic, 22,343.667 for baseline DQN and 22,046.367 for penalty-tuned DQN. Under stress, the corresponding means are 18,444.400, 18,333.207 and 18,237.027. The penalty-tuned policy reduces mean normal lost sales from 5.020 to 2.647 units, while increasing monthly holding cost from 1,249.333 to 1,760.233. Thus, the tested improvement succeeds on shortage reduction but not on original-cost profit. All monetary values are simulation units rather than a specified real currency.

The executed notebook preserves successful tests, training logs, tables and four figures. Its output records creation of saved models and a results archive, but the separate archive and model files are not currently available in the published repository. The environment is a custom single-store simulator, not the assignment's required InvManagement-v1. This report does not claim that this substitution has instructor approval.

Report structure

Sections	Coverage
1–2	Assignment scope and comparison with the supplied reference
3–5	Environment, inventory accounting and baseline policy
6–8	DQN, penalty modification and learning evidence
9–12	Evaluation, normal and stress results, uncertainty
13–15	Interpretation, reproducibility, conclusions and sources

Introduction

Retail inventory decisions must balance two competing risks. Ordering too much ties up stock and increases holding costs; ordering too little leaves customer demand unmet. Replenishment lead time makes this harder because an order placed today cannot immediately replace inventory that has already sold out. Under random demand, a useful controller must make these decisions before the next customer-demand quantity is known.

This project studies that problem through a 30-day, single-product inventory simulation. Its objective is to compare a traditional order up to policy with a Deep Q-Network that learns daily order quantities from inventory, delivery-pipeline and remaining-time information. A second DQN uses a tenfold stockout penalty during training to investigate whether stronger shortage aversion improves service and how that change affects holding cost and profit.

Project objectives

The study implements and tests the inventory environment, establishes a validation-selected (s, S) benchmark, trains baseline and penalty-tuned DQNs under equal budgets, and evaluates all policies on matched unseen demand scenarios. It also introduces a three-day demand spike to examine the effect of delivery delay and the policies' recovery behaviour. The analysis distinguishes original-cost profit from the modified training reward and reports both aggregate performance and variation across training seeds.

Code and executed notebook

The project repository contains the executed notebook, runnable experiment source, environment tests, dependency requirements and supporting documentation. The results and figures discussed in this report are grounded in that executed notebook. The repository is available at:

https://github.com/virat-mankali/reinforcement-learning-project-virat-mankali

Scope of the report

The report follows the five Project 5 tasks, compares the implementation with the supplied reference notebook, explains the model and learning procedure, and interprets normal-demand and stress-test results. The study uses a custom Gymnasium simulator rather than the specified or-gym environment. This difference, and the current availability of saved-model artifacts, are stated explicitly so that the reported evidence is not mistaken for full compliance with every submission requirement.

1 Assignment scope and completion

Project 5 asks for an inventory controller that balances holding costs against lost-sales penalties under random customer demand. The required experiment contains a heuristic baseline, RL training, one improvement, 100-month testing, a three-day demand spike and an explanation of the learned policy, lead time and aleatoric uncertainty. [1]

Required task	Evidence in this report	Status
1 Environment and heuristic	Sections 3–5; setup, (s, S) rule, daily profit, holding and stockout metrics	Custom environment deviation
2 Baseline RL	Sections 6 and 8; DQN setup, 150,000 steps, training plots	Executed; model delivery gap
3 One improvement	Section 7; exact 10× penalty logic and retraining curves	Executed
4 Formal and stress evaluation	Sections 9–11; 100 months per condition, metrics and inventory time series	Executed
5 Interpretation	Sections 12–13; mathematical policy comparison, lead time, uncertainty and failure modes	Included

Important environment deviation

The brief explicitly requests or-gym InvManagement-v1. The supplied reference uses a custom Gymnasium environment and describes it as the same MDP. This report does not adopt that equivalence claim. The or-gym implementation represents a multi-echelon supply chain and uses a Box action space, whereas this study models one store and uses a discrete action grid. The assignment's simplified discrete-action description does not remove this difference. [1, 5]

A custom environment makes the transition rules and accounting inspectable, but it does not fulfil the named-environment requirement by itself. If literal use of InvManagement-v1 is mandatory, the study must be migrated or an exception must be approved. The existing results apply only to the custom model.

Evidence boundary

Numerical findings come from the completed notebook, rather than a fresh local training run. The saved cell outputs show 11 executed code cells without error outputs, 84 passing environment tests, six completed training runs and the expected evaluation groups. Figures in this report are extracted directly from those outputs. [2]

2 Comparison with the supplied reference

The supplied notebook, RL_Inventory_Management_FINAL (1).ipynb, was identified by the student as Ajay Simha Reddy's reference work. It provides the task-by-task structure and a useful example of a custom inventory simulator. It was reviewed as a reference; its saved outcomes were not rerun or presented as this project's results. Authorship and reuse permission have not been independently established. [3]

Design choice	Supplied reference	This executed study
Environment	Single-product custom Gymnasium	Custom Gymnasium; difference from or-gym disclosed
State	Raw inventory, pipeline, last demand, day; extra 5-day mean	Scaled inventory, separate pipeline buckets, time remaining
Orders	121 integer quantities, 0–120	25 requested quantities in five-unit increments
Improvement	Moving-average demand feature	10× stockout penalty during training
Training	One seed; 150,000 steps per variant	Three seeds per variant; same 150,000-step budget
Heuristic	Fixed s = 60, S = 180	Validation-selected s = 80, S = 80
Evaluation	Different heuristic and RL seed offsets	Common demand bank for all policies
Accounting	Free opening stock; no terminal salvage; arrival clipping	Opening stock purchased; terminal salvage; capacity reserved
Stress evidence	One illustrated shock episode	100 stress months plus a fixed example

The reference's saved final table reports average daily profit of 564.17 for the heuristic, −2,622.97 for baseline DQN and 591.99 for the moving-average variant. These are reference-only outputs. The large negative baseline result warrants investigation; it cannot be treated as proof that adding a moving average caused the entire improvement. The accounting, state scaling, policies and evaluation design differ from this study.

In the reference simulator, daily demand is IID Poisson with fixed mean 20. A trailing average is therefore not a genuine forecast of tomorrow's independent demand. It may change numerical learning behaviour, but it does not reveal a persistent demand trend in that model. Choosing the rubric's penalty-tuning option gives a clearer experiment on the service-versus-cost trade-off.

3 Environment and problem formulation

The environment is a finite-horizon lost-sales inventory problem. The agent chooses an order before seeing that day's demand. Stock ordered today becomes available before demand two days later. Unfulfilled demand is lost immediately and is not carried forward as a backlog. [2]

Parameter	Value and meaning
Episode horizon	30 daily decisions
Normal demand	Independent Poisson draws with mean 20 units per day
Stress demand	Mean 60 on days 15–17; mean 20 otherwise
Lead time	2 days, deterministic
Opening inventory	40 units on hand; no pipeline stock
Reserved capacity	250 units across on-hand stock and all pipeline buckets
Requested order	0, 5, 10, ..., 120 units; Discrete(25)
Selling price and purchase cost	100 and 60 simulation monetary units per unit
Holding cost	2 per unit of end-of-day on-hand stock per day
Original shortage penalty	50 per unit of lost demand
Penalty-tuned training	Shortage penalty 500 per lost unit
Terminal treatment	On-hand and pipeline stock credited at purchase cost

Observation and state information

Before a decision, let I_t denote on-hand inventory, P_t^0 the pipeline quantity due today, P_t^1 the quantity due tomorrow, and t the number of completed days. The observation is $x_t = (I_t/250, P_t^0/250, P_t^1/250, (30 - t)/30)$. Its declared space is Box(0, 1, shape=(4,), dtype=float32). The initial observation is [0.16, 0, 0, 1].

Each pipeline bucket is included separately because arrival timing matters. Time remaining allows a finite-horizon policy to distinguish early and late episode decisions. Under the stationary IID training distribution, past demand is not needed to predict the next demand distribution. Omitting historical demand departs from the brief's state description. During stress, the demand distribution shifts outside the training distribution; no warning variable is supplied.

4 Inventory transitions and profit accounting

Daily transition order

For action index a_t, the requested order is $q_t = 5a_t$. The environment accepts $u_t = \min(q_t, 250 - I_t - P_t^{(0)} - P_t^{(1)})$. It then receives $P_t^{(0)}$, shifts the pipeline to $(P_t^{(1)}, u_t)$, observes demand D_t, sells $Y_t = \min(I_t + P_t^{(0)}, D_t)$, and records lost units $L_t = D_t - Y_t$. End-of-day inventory is $I_{t+1} = I_t + P_t^{(0)} - Y_t$.

Capacity is checked before arrivals and demand. Reserving room for every outstanding order is conservative, since later sales might free space, but it prevents paid inventory from disappearing when a shipment arrives. Accepted orders can be smaller than the five-unit request grid when capacity binds. Rejected quantities are recorded explicitly and are not purchased.

Original economic reward

Daily original-cost profit is defined as follows, with t = 0 on the first day:

$$\pi_t = 100Y_t - 60u_t - 2I_{t+1} - 50L_t \\ - 60I_0 \times 1[t = 0] + 60(I_{30} + \Sigma P_{30}) \times 1[t = 29]$$

The opening inventory charge is 60 × 40 = 2,400. Terminal salvage values both unsold on-hand stock and outstanding pipeline stock at unit purchase cost. This avoids treating opening stock as free or treating every late order as a permanent loss merely because the simulation ends. The assumption is idealised: a real retailer may not recover full value for excess inventory or undelivered orders.

A useful ledger identity

Inventory conservation gives $I_0 + \Sigma u_t = \Sigma Y_t + I_{30} + \Sigma P_{30}$. Consequently, full-month profit simplifies to $40\Sigma Y_t - 2\Sigma I_{t+1} - 50\Sigma L_t$. This is a check on the model's accounting, not a replacement for the daily ledger used by the agent. No discounting is used, so these terminal credits are not distorted by time preference.

The simulator tests cover seeded reproducibility, observations, exact delivery timing, sales conservation, capacity, opening and terminal accounting, penalties, invalid inputs and episode boundaries. The executed Colab output reports 84 passed in 5.19 seconds. These checks support implementation consistency; they do not establish that the assumptions are realistic or that the environment matches or-gym. [2]

5 Order up to heuristic baseline

The traditional comparator uses inventory position $J_t = I_t + P_t^0 + P_t^1$. If J_t is below reorder threshold s, it requests enough stock to approach S; otherwise, it requests zero. Using inventory position prevents repeated ordering that ignores shipments already on their way.

$q^*_t = \min(120, \max(0, S - J_t))$ when $J_t < s$; otherwise $q^*_t = 0$.
a_t = arg min over $a \in \{0, \ldots, 24\}$ of $|5a - q^*_t|$.

The same capacity rule then limits the accepted order. The nearest grid action is chosen; NumPy's argmin selects the first minimum in a tie. Both the heuristic and the DQNs therefore operate through the same actions and physical constraints.

Validation selection

Thresholds are selected on 50 independent normal-demand validation months. The search considers s from 30 to 100 in steps of 10 and S from max(50, s) to 140 in steps of 10. The pair with the highest mean validation profit is retained. This procedure chooses s = 80 and S = 80. The search does not use final normal or stress test outcomes.

With s equal to S, the selected rule behaves as a base-stock policy: whenever inventory position is below 80, it replenishes towards 80. This is a valid boundary case of the threshold rule. It should not be confused with ordering to physical capacity 250. The target is an economically chosen inventory position, while 250 is the model's safety constraint.

Measured baseline performance

Metric	Normal months	Stress months
Mean monthly profit	22,615.980	18,444.400
Mean daily profit	753.866	614.813
Mean monthly holding cost	1,166.020	1,059.000
Mean daily holding cost	38.867	35.300
Mean stockout days per month	0.630	3.450
Total stockout days in 100 months	63 / 3,000 days	345 / 3,000 days
Mean lost units per month	2.920	103.340

The daily holding values are monthly means divided by 30; total stockout days are derived from the reported means and 100-month evaluation count. A stockout event is a day with positive unmet demand, rather than each individual lost unit. Under stress, lower holding cost does not signal better inventory control: stock can be depleted because customers cannot be served.

6 Baseline Deep Q Network

DQN is appropriate here because the action space contains 25 discrete order quantities. A neural network maps the four-dimensional observation to one action value per order. During deterministic evaluation, the controller selects $a_t = \arg\max_a Q\theta(x_t, a)$. Stable-Baselines3 implements vanilla DQN using replay memory and a target network; this experiment does not claim Double DQN or a recurrent architecture. [4]

For a sampled transition, the one-step target is $y = r + \gamma(1 - \text{done}) \max_{a'} Q\theta^{-}(x', a')$. The network learns to bring $Q\theta(x, a)$ closer to this target. Replay reuses earlier transitions; the target network slows changes in the bootstrap target. Both reward variants use the same learning configuration. [2, 4]

Setting	Executed configuration
Policy network	MlpPolicy; two hidden layers of 64 units
Training steps	150,000 per model; 5,000 complete 30-day episodes
Training seeds	11, 22, 33 for each reward variant
Learning rate	0.0003
Replay buffer and warm-up	50,000 transitions; learning begins after 3,000 steps
Minibatch and updates	128 samples; one gradient step every 4 environment steps
Discount factor	$\gamma = 1.0$
Target network interval	1,000 environment steps
Exploration	Initial $\varepsilon = 1.0$; reduced to 0.05 over first 50% of training
Reward scaling	Daily objective divided by 1,000
Execution	CPU; one PyTorch thread
Selection	Original-profit validation every 25,000 steps

With a finite 30-day horizon and time remaining in the state, $\gamma = 1$ aligns the accumulated training objective with undiscounted monthly profit. The terminal transition ends bootstrapping. Dividing reward by 1,000 changes numerical scale, not the ordering of policies under the same economic objective.

Across six runs, the study uses 900,000 training steps. An additional 10,000-step readiness benchmark is discarded and is not a candidate model. The executed log records a 25.1-second benchmark and a 16.7-minute full experiment. The benchmark's rough 52.7-minute projection was an estimate, not the measured final runtime.

7 Improvement through penalty tuning

The selected improvement is the assignment's tenfold stockout-penalty option. The hypothesis is that a larger loss signal will encourage a more conservative policy and reduce unserved demand. This is tested as a candidate improvement rather than assumed to improve every metric.

Exact modification from the executed source

```
stockout = self.stockout_penalty * lost
business_profit = revenue - purchase - opening - holding - stockout + salvage
shaped_profit = business_profit - (self.penalty_multiplier - 1) * stockout
return self._obs(), shaped_profit / 1000., terminated, False, info
```

The baseline uses penalty_multiplier = 1.0; the second variant uses 10.0. Therefore r_t(base) = π_t/1,000 and r_t(penalty) = (π_t − 450L_t)/1,000. The physical transition rules, action grid, observations, network, seeds and training budget are unchanged.

Why final evaluation uses the original costs

A shaped return is not directly comparable with an unshaped return because the shortage prices differ. All final policy evaluations instantiate the original-cost environment with a shortage penalty of 50. The report therefore compares the same financial quantity across all policies while separately showing the shaping objective during training.

Checkpoint selection also uses original-cost mean validation profit. This tests whether a shortage-averse training signal can produce a useful policy for the original business objective. It does not select the checkpoint with the lowest stockout count. A service-constrained selection rule would be a different experiment and should be declared before looking at test outcomes.

Expected trade off

Holding more inventory reduces exposure to random demand but costs 2 per retained unit per day. A missed sale also loses its 40-unit contribution margin and incurs the original 50-unit shortage penalty. Excessive safety stock can therefore erase the benefit of fewer shortages. The experiment reports this trade-off explicitly instead of relabelling a profit regression as success.

The feature-engineering option was not added alongside penalty tuning. Under the IID demand model, a five-day average does not predict a trend. Keeping the observation unchanged also isolates the reward intervention between the two DQN variants.

8 Training and validation evidence

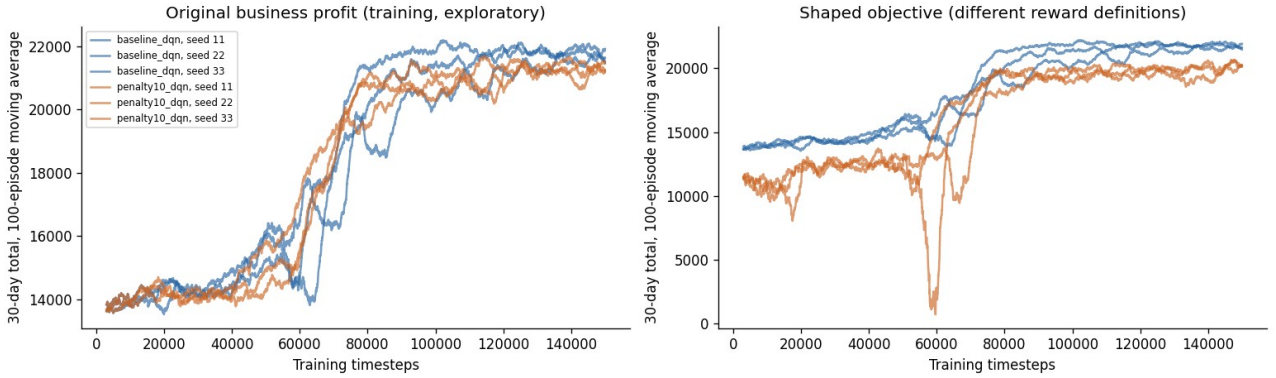


Figure 1. Executed training curves for all three seeds of each DQN variant. Curves show 100-episode moving averages of unscaled 30-day totals. The left panel uses common original costs; the right panel uses different shaping objectives.

Original-cost training profit generally improves after approximately 60,000–80,000 steps, with visible instability and seed differences before settling near the final range. The penalty variant's shaped curves exhibit particularly deep temporary declines. These exploratory training returns include random actions and must not be substituted for deterministic test performance.

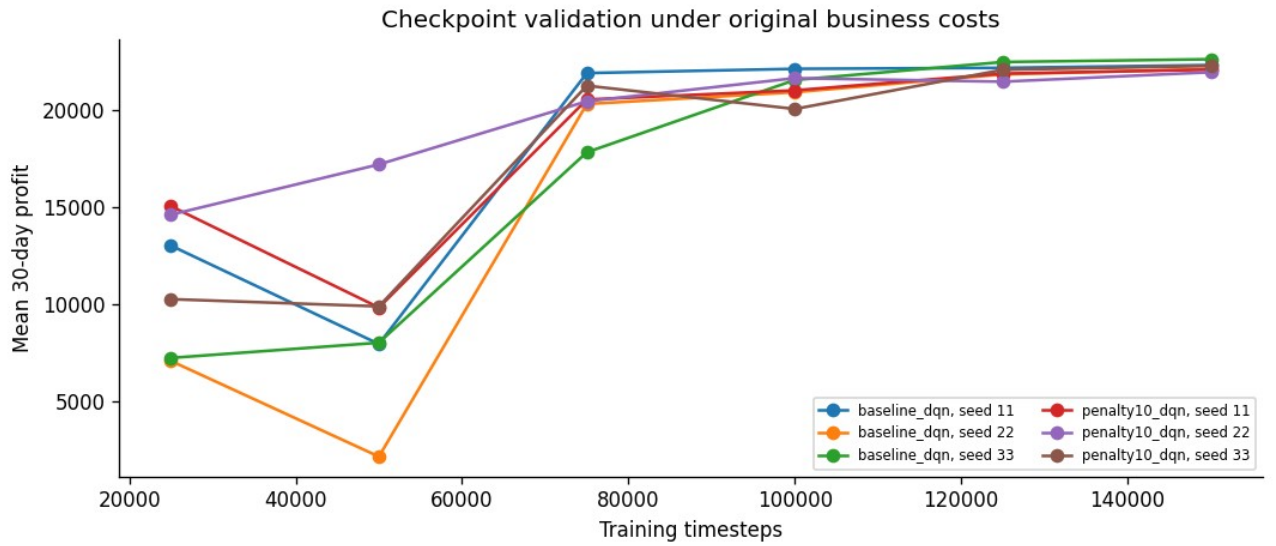


Figure 2. Deterministic checkpoint validation on 50 normal-demand months under original costs. Checkpoints are evaluated every 25,000 steps; test months are not used for selection.

Validation makes the early instability especially clear: baseline seed 22 falls from 7,070.6 at 25,000 steps to 2,144.1 at 50,000, then rises to 20,298.8 at 75,000. The same model reaches 22,096.6 at 150,000. A single early or attractive checkpoint would give a misleading account of training. The source saves both final and best-validation checkpoints, and evaluates the saved best model for each seed. In this run, every model's highest recorded validation profit occurs at 150,000 steps, so the selected and final checkpoints coincide. [2]

9 Evaluation design and metric definitions

Training samples fresh IID demand through each environment's random generator. A separate validation bank contains 50 months generated from seed 10100. Final normal and stress banks contain 100 independent months each, generated from seeds 20200 and 30300. SeedSequence creates independent child streams within each bank. The stress distribution triples its mean on zero-based indices 14–16, corresponding to displayed days 15–17.

Every policy and training seed receives exactly the same demand sequence for each episode within a condition. This common-random-number design makes policy differences less sensitive to which demand draws happened to be sampled. The normal and stress banks are distinct, so their difference is not a paired causal estimate of the shock on an otherwise identical month.

Measure	Definition and aggregation
Monthly profit	Sum of the 30 daily original-cost profit values
Average daily profit	Monthly profit divided by 30
Monthly holding cost	Sum of 2 × end-of-day inventory
Average daily holding	Monthly holding cost divided by 30
Lost units	Sum of demand minus sales over the month
Stockout days	Count of days for which lost units are greater than zero
Stockout day rate	Stockout days divided by 30; not the fraction of units lost
Fill rate	Sales divided by demand within each month, then averaged
RL summary	Mean over 100 months and three trained seeds
Heuristic summary	Mean over 100 months for one selected deterministic rule

Three RL seeds evaluated on 100 shared demand sequences produce 300 policy outcomes per variant and condition, but only 100 independent demand scenarios. They must not be described as 300 independent demand months. The heuristic has 100 outcomes per condition, giving 700 policy outcomes per condition and 1,400 overall.

Deterministic prediction disables exploration during testing. The first test month and training seed 11 are fixed for the representative plot; the figure is not chosen for attractive performance. Reported tables retain the notebook's precision. Rates rounded to three decimals should not be interpreted as exact probabilities.

10 Results under normal and stress demand

Normal demand metric	Heuristic	Baseline DQN	10× penalty
Monthly profit	22,615.980	22,343.667	22,046.367
Daily profit	753.866	744.789	734.879
Monthly holding	1,166.020	1,249.333	1,760.233
Daily holding[1]	38.867	41.644	58.674
Lost units per month	2.920	5.020	2.647
Mean fill rate	0.995	0.992	0.996
Stockout days per month	0.630	1.253	0.567
Stockout day rate	0.021	0.042	0.019

Stress demand metric	Heuristic	Baseline DQN	10× penalty
Monthly profit	18,444.400	18,333.207	18,237.027
Daily profit	614.813	611.107	607.901
Monthly holding	1,059.000	1,096.093	1,795.873
Daily holding[1]	35.300	36.536	59.862
Lost units per month	103.340	104.163	97.457
Mean fill rate	0.857	0.856	0.865
Stockout days per month	3.450	3.990	3.290
Stockout day rate	0.115	0.133	0.110

[1] Daily holding values are calculated from the displayed monthly totals divided by 30. All costs and profit use simulation monetary units. All other values are transcribed from the executed summary. [2]

The heuristic has the highest mean original-cost profit in both conditions. Compared with baseline DQN, penalty tuning reduces normal lost units by approximately 47.3% but raises holding cost by 40.9%. Under stress, it reduces lost units by approximately 6.4% while raising holding cost by 63.8%. These relative changes are calculated from the reported means.

The mean profit reductions are 297.300 in normal months and 96.180 in stress months. Because demand is matched within each condition, the ledger explains the result: each additional served unit improves original-cost contribution by 90, while extra holding reduces profit. The normal gain from approximately 2.373 additional served units is about 213.6, less than the extra holding cost of 510.9. Stress gains of about 603.6 are also below extra holding of 699.78. Small rounding differences arise from displayed precision.

11 Stress behaviour and visual comparison

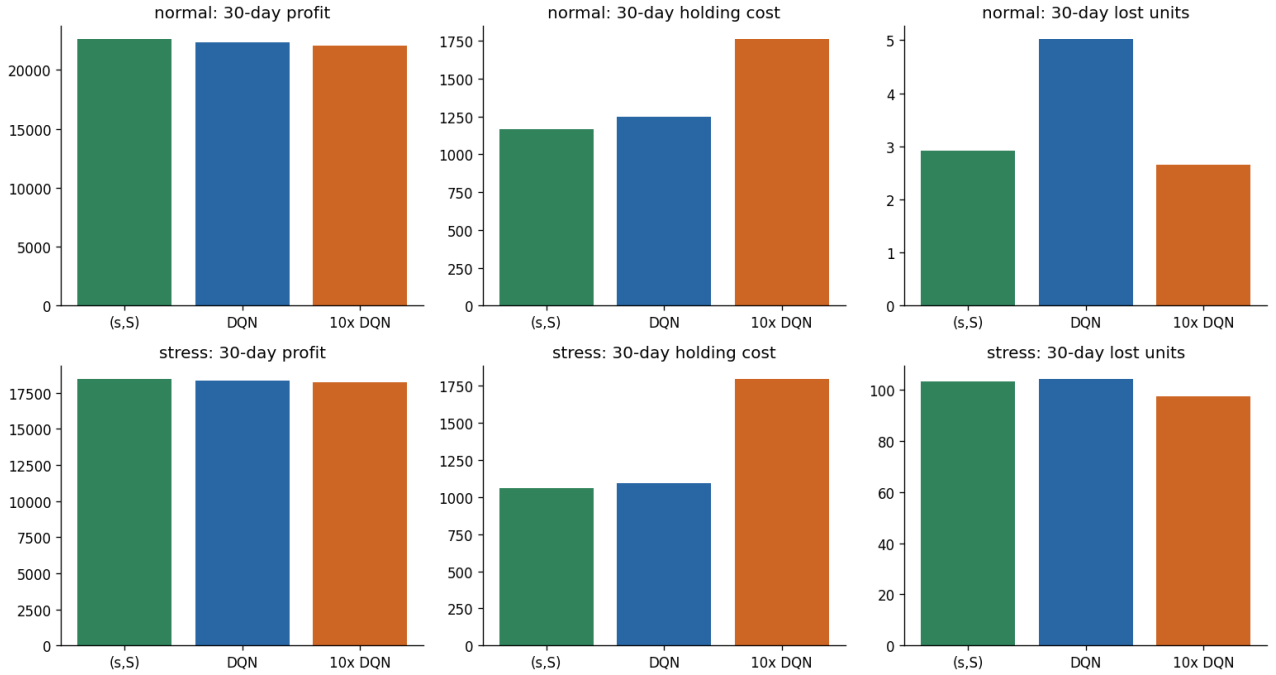


Figure 3. Genuine notebook comparison of mean profit, holding cost and lost sales. Top row: normal demand. Bottom row: three-day demand stress. Bar heights are means, without uncertainty intervals.

The comparison shows why profit and service must be reported together. The penalty-tuned policy has fewer mean lost units in both conditions but maintains more inventory, producing the largest holding costs. The stress case reduces all policies' mean profit and fill rates substantially. None achieves zero stockouts.

Interpreting the shock

The mean rises from 20 to 60 for three days, adding 120 expected units over a normal month. This is a tripling of the Poisson rate, not multiplication of each realised normal demand value by three. The trained agent receives no promotion flag and was not trained on recurring spikes. It can respond to depleted stock and pipeline state, but cannot infer the exact unseen demand before it occurs.

The next page shows the full inventory, in-transit stock and accepted-order response for the fixed example. Stock remaining at the end of a day is distinct from the quantity available before demand. A plotted zero means all available stock was consumed; the separate lost-unit metric establishes whether customers were actually turned away.

11 Stress trajectory continued

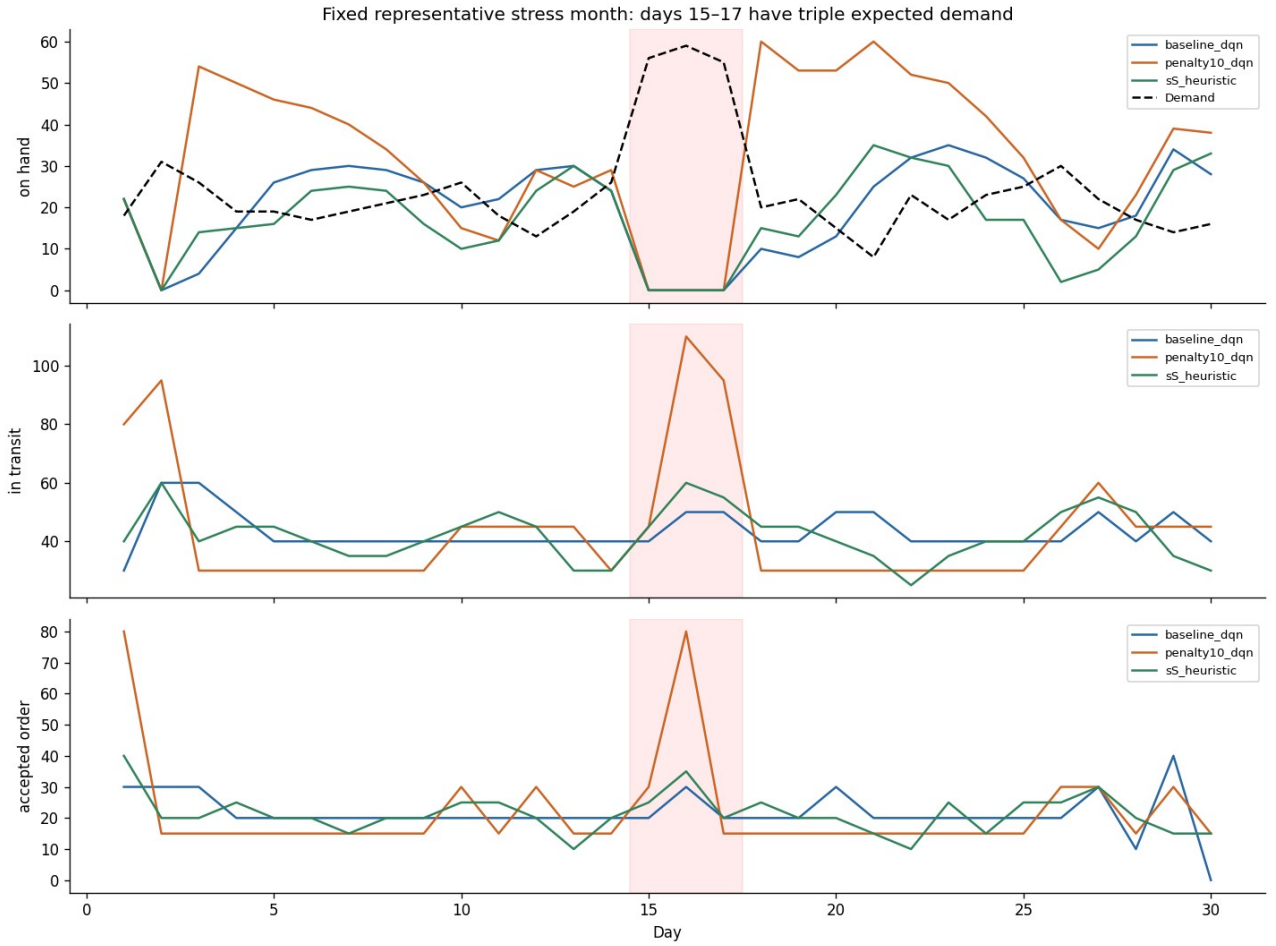


Figure 4. Fixed stress test episode 0, using training seed 11 for both DQNs. The shaded days 15–17 have triple expected demand. Pipeline values sum both outstanding-order buckets.

In this example, all three policies end days 15–17 with zero on-hand stock. The penalty-tuned DQN raises its accepted order to 80 units on day 16; that order arrives on day 18 and produces a pronounced inventory rebound after demand returns to its normal distribution. Its later stock levels are higher than the other policies for much of the recovery period, consistent with greater holding cost.

The policies resume positive inventory after the spike rather than remaining at zero for the rest of the month. This is recovery in one fixed trajectory, not evidence that every stress episode recovers equally quickly. The 100-month stress results still show substantial lost sales, including 97.457 units per month for the penalty-tuned variant. No formal average recovery-time metric was collected.

12 Variability and statistical comparison

The aggregate result should be read alongside training-seed variation. The table reports mean original-cost profit over the same 100 test months for each trained model. It shows that the ranking of individual seeds is not identical to the ranking of three-seed averages.

Training seed	Normal baseline	Normal 10×	Stress baseline	Stress 10×
11	22,269.60	22,012.52	19,044.12	17,630.74
22	22,163.90	21,913.84	18,037.74	18,438.84
33	22,597.50	22,212.74	17,917.76	18,641.50

For example, baseline seed 11 outperforms the heuristic on mean stress profit, whereas the baseline average across all three seeds does not. Stress profit for baseline DQN spans 17,917.76–19,044.12 across seeds. With only three seeds, this study cannot establish broad training stability or a universal policy ranking.

Paired demand scenario bootstrap

For each test month, the three RL seed profits are first averaged. A paired difference is then computed against the heuristic, or between RL variants, on that same demand scenario. The notebook resamples 100 episode-level differences with replacement 5,000 times and uses the 2.5th and 97.5th percentiles of the mean difference. Bootstrap seed 40400 makes this calculation reproducible.

Condition and comparison	Mean difference	95% interval
Normal baseline − heuristic	−272.313	[−319.487, −230.092]
Normal 10× − heuristic	−569.613	[−600.556, −537.620]
Normal 10× − baseline	−297.300	[−355.461, −234.459]
Stress baseline − heuristic	−111.193	[−195.641, −25.166]
Stress 10× − heuristic	−207.373	[−271.569, −141.589]
Stress 10× − baseline	−96.180	[−187.075, −3.555]

These intervals describe demand-scenario uncertainty conditional on the six fitted models. They do not incorporate full training-seed uncertainty, hyperparameter-search uncertainty or model misspecification. Their negative bounds support lower mean profit for the displayed comparisons on this experiment, rather than proving that every retrained DQN would perform worse. No confidence intervals for service metrics are inferred from the profit intervals.

13 Interpretation and failure modes

Learned policy compared with the heuristic

The heuristic compresses the decision into two thresholds and total inventory position. DQN instead approximates action values over on-hand stock, each delivery bucket and remaining time. Two states with the same inventory position can receive different DQN actions if deliveries arrive at different times. This flexibility can matter, but it also introduces approximation error and a harder optimisation problem. In this stationary single-product experiment, the simpler validation-tuned rule remains the strongest mean-profit comparator.

How lead time changes the decision

An order cannot repair a shortage on the same day. With a two-day delay, adequate stock and already-placed orders must cover intervening demand. The observation exposes both delivery buckets so the DQN can learn this delayed consequence. The stress example shows a larger order followed by a delayed rebound, rather than immediate shortage prevention. It is evidence of a lead-time response, not proof that the network has discovered an optimal replenishment law.

Aleatoric uncertainty

Poisson demand is random even when its distribution is known. For independent demand, $E[D_{t+1} \mid \text{past demand}] = 20$ during normal operation, but the next realised draw still varies. More training can reduce estimation and optimisation uncertainty, but cannot remove demand randomness. Standard expected-return optimisation trades shortage exposure against holding costs; a larger penalty shifts that trade-off without guaranteeing service. With unbounded Poisson demand and finite inventory, a strict guarantee of no lost sales is impossible.

Observed and potential failure modes

Observed: training has temporary performance drops; outcomes differ across seeds; all policies suffer shortages during the example spike; and stronger penalties create expensive post-spike inventory. Each episode starts with only 40 units and no pipeline, so an unusually high initial demand can cause shortages before any new order arrives.

Potential: full-cost terminal salvage may encourage late orders with little economic downside; conservative capacity reservation may reject orders that a physical warehouse could later accept; unknown demand trends or random supplier delays would invalidate the training assumptions. The finite 30-day reset is not a continuous retail operation, and the model omits spoilage, order setup fees, multiple products and supplier capacity risk.

Deployment would require real demand and cost calibration, service constraints, operational review and monitoring. The simulation supports a comparison of learning objectives, not a claim of real retail profitability or a safe autonomous purchasing system.

14 Reproducibility and submission evidence

Available evidence

The repository contains the executed notebook, inventory_experiment.py, test_inventory.py, pinned requirements and README. The notebook contains the genuine training logs, final metrics, seed table, paired intervals and all four report figures. Its final output records creation of a 2.2 MB retail_inventory_results.zip archive. [2]

The saved-model requirement is only partially evidenced: the code saves best and final models for every run and reloads best checkpoints during the completed experiment, but the separate model ZIP files, raw CSV tables, runtime manifest and results archive have not been recovered into the published repository. They are not attached to this report. Visible notebook outputs do not substitute for executable model artifacts.

How to reproduce in Google Colab

1. Open the published self-contained notebook in a fresh Colab CPU runtime. The notebook writes the source and tests needed for the experiment.
2. Run its pinned setup cell, then the environment checks. Confirm the tests pass before proceeding.
3. Run the readiness benchmark, followed by six training runs with seeds 11, 22 and 33 and 150,000 steps per run.
4. Preserve validation selection and the fixed test banks. Run the evaluation and plotting cells without changing parameters after inspecting test results.
5. Download the results archive and executed notebook separately. Verify model ZIPs, raw metrics, demand banks and manifest before treating a new run as a complete reproducibility package.

Dependency	Pinned version
PyTorch CPU	2.7.1
Stable-Baselines3	2.7.0
Gymnasium	1.2.0
NumPy / pandas	2.3.5 / 2.2.3
Matplotlib / pytest	3.10.8 / 8.4.2

These are the dependency pins in the supplied executed project, not a claim about the latest releases. A fresh run creates new artifacts and can produce different numerical outcomes. Hardware details and exact per-model runtime metadata should be taken from a recovered or newly generated manifest; they are not invented here.

Remaining submission actions

Obtain approval for the environment substitution or migrate to the specified environment. Recover and verify the original saved models and raw results if still available; otherwise perform a clearly labelled new Colab run and preserve its artifacts. These gaps are separate from the completed analysis in this report.

15 Conclusion and references

This project demonstrates a complete executed comparison of a classical replenishment rule and two DQN reward variants in a transparent custom inventory simulator. Both RL variants learn useful ordering behaviour, but the validation-selected (s, S) heuristic achieves the highest mean profit under normal and stress demand. The tenfold shortage penalty reduces unserved demand and stockout-day frequency while increasing holding costs enough to reduce original-cost profit.

The main lesson is that an improvement must be judged against its stated objective. Penalty tuning improves service in this experiment, but does not improve profit or eliminate shortage risk. Common test scenarios, multiple training seeds, explicit inventory accounting and separate validation make this conclusion more credible than selecting one favourable run. The custom-environment deviation and missing standalone models remain material limitations of assignment compliance.

References and evidence sources

[1] Course assignment. ProjectDetails.pdf, Project 5, pages 10–11. Supplied assignment brief; environment, tasks and required outputs.

[2] Virat Mankali. Retail_Inventory_DQN_Reproducible_Study.ipynb and accompanying source, tests and README. Executed notebook used for all current-study results and figures. Repository revision 8e7236c8f9f8b2a97cc5c031934329df01d835f6.

Repository
https://github.com/virat-mankali/reinforcement-learning-project-virat-mankali

Executed notebook in the cited revision
https://github.com/virat-mankali/reinforcement-learning-project-virat-mankali/blob/8e7236c8f9f8b2a97cc5c031934329df01d835f6/Retail_Inventory_DQN_Reproducible_Study.ipynb

[3] Supplied reference notebook. RL_Inventory_Management_FINAL (1).ipynb, identified by the student as Ajay Simha Reddy's work. Reviewed for implementation and task coverage. Its saved results are reference-only and were not rerun for this report.

[4] Stable-Baselines3. DQN documentation, version 2.7.0. Consulted for the algorithm implementation, supported action spaces and configuration definitions.
https://stable-baselines3.readthedocs.io/en/v2.7.0/modules/dqn.html

[5] OR-Gym project. inventory_management.py, supply-chain environment source. Consulted to distinguish the specified environment from the custom single-store model.
https://github.com/hubbs5/or-gym/blob/master/or_gym/envs/supply_chain/inventory_management.py

Web documentation accessed 14 October 2026. No reference-only performance number is included in the current-study comparison tables.